

Embeddings

Nhóm 2

Nguyễn Hồng Phúc - 24521390

Vũ Minh Phương - 24521421

Võ Lê Ngọc Thịnh - 24521710

Nguyễn Duy Khang - 24520755

University of Information Technology, VNU-HCM

Faculty of Computer Science

CS221.Q21.KHTN

17/03/2026

- 1 Word2vec
- 2 Visualizing Embeddings
- 3 Semantic properties of embeddings
- 4 Bias and Embeddings
- 5 Evaluating Vector Models

- 1 Word2vec
- 2 Visualizing Embeddings
- 3 Semantic properties of embeddings
- 4 Bias and Embeddings
- 5 Evaluating Vector Models

Motivation: Sparse Vectors and Limitations

1. Sparse Vectors

- **Kích thước ($|V|$):** Rất dài, tương đương số lượng từ vựng trong từ điển, thường từ 10.000 đến 50.000 chiều.
- **Tính chất:** Hầu hết các giá trị là 0, tạo thành các sparse vectors.

2. Hạn chế cốt lõi

- Khó nắm bắt được tính đồng nghĩa (synonymy).
- **Ví dụ:** Các chiều đại diện cho các từ đồng nghĩa như *car* và *automobile* là hoàn toàn tách biệt và không liên quan. Dẫn đến việc mô hình thất bại trong việc nhận diện sự tương đồng giữa hai từ có ngữ cảnh lân cận là *car* và *automobile*.

Motivation: Dense Vectors

1. Dense Vectors

- **Kích thước (d):** Ngắn và dày đặc, số chiều thường chỉ từ 50 đến 1000.
- **Tính chất:** Các giá trị là số thực và có thể mang giá trị âm.

2. Tại sao Dense Vectors tốt hơn?

- Hoạt động hiệu quả hơn trong mọi tác vụ NLP so với sparse vectors.
- **Tối ưu Parameter:** Yêu cầu classifier học ít trọng số hơn rất nhiều.
- **Generalization:** Không gian tham số nhỏ giúp mô hình tổng quát hóa tốt hơn và tránh overfitting.

Word2vec & Skip-gram: Giới thiệu chung

1. Word2vec là gì?

- Là một gói phần mềm (software package) chứa các thuật toán học embedding.
- **Skip-gram with Negative Sampling (SGNS)** là một thuật toán của word2vec.
- **Ưu điểm:** Nhanh, huấn luyện hiệu quả và dễ dàng tìm thấy các pretrained embeddings có sẵn.

2. Tính chất: Static Embeddings

- Word2vec tạo ra các phép nhúng tĩnh (static embeddings).
- **Đặc điểm:** Phương pháp này học và gán một vector cố định duy nhất cho mỗi từ trong từ vựng.

Ý tưởng cốt lõi của Word2vec

1. Ý tưởng trực quan

- Thay vì đếm tần suất đồng xuất hiện, ta huấn luyện một classifier cho bài toán binary prediction.
- **Yêu cầu:** Trả lời câu hỏi "*Từ c có khả năng xuất hiện gần từ đích w hay không?*".
- **Mục tiêu:** Không quan tâm đến bản thân tác vụ dự đoán, mà lấy các trọng số đã học từ classifier làm word embeddings.

Self-supervision - Ý tưởng đột phá

Sử dụng chính văn bản thô (running text) làm dữ liệu huấn luyện giám sát ngầm định. Từ c xuất hiện gần từ đích w được coi là *gold* "*correct answer*". Cách này loại bỏ hoàn toàn việc cần con người dán nhãn thủ công.

Skip-gram Algorithm

Thuật toán Skip-gram đơn giản hóa quá trình học thông qua 4 bước cơ bản sau:

- ➊ **Positive Examples:** Lấy từ đích (target word) và một từ ngữ cảnh lân cận (neighboring context word) làm các ví dụ dương.
- ➋ **Negative Samples:** Lấy mẫu ngẫu nhiên các từ khác trong từ vựng để tạo các ví dụ âm.
- ➌ **Train Classifier:** Sử dụng logistic regression để huấn luyện một bộ phân loại nhằm phân biệt hai trường hợp trên.
- ➍ **Extract Embeddings:** Sử dụng các trọng số đã học được làm các phép nhúng embeddings cho từ.

Mathematics of the Classifier: The Classification Task

1. Nhiệm vụ phân loại

- Xét một câu ví dụ với target word $w = \mathbf{apricot}$ và context window là ± 2 từ:
*... lemon, a [tablespoon_{c1} of_{c2} **apricot**_w jam_{c3}, a_{c4}] pinch ...*

2. Mục tiêu của bộ phân loại

- Nhận đầu vào là một tuple (w, c) gồm target word w và một candidate context word c .
- Mục tiêu:** Trả về xác suất c là một real context word của w .
- Ví dụ:** Cặp $(apricot, jam)$ sẽ trả về xác suất cao (True), trong khi cặp ngẫu nhiên $(apricot, aardvark)$ sẽ trả về xác suất thấp (False).

Công thức xác suất cơ bản

- Xác suất c là từ ngữ cảnh của w : $P(+|w, c)$
- Xác suất c không phải là từ ngữ cảnh của w : $P(-|w, c) = 1 - P(+|w, c)$

Độ tương đồng của embeddings (Embeddings Similarity):

- Skip-gram dựa trên trực giác: Một từ dễ xuất hiện gần target word nếu vector embedding của nó tương đồng với embedding của target word.
- Để tính similarity giữa hai dense embeddings, ta dùng **dot product**:

$$\text{Similarity}(w, c) \approx c \cdot w \quad (5.13)$$

Mathematics of the Classifier: Computing Probability P

Chuyển đổi dot product thành xác suất:

- Dot product $c \cdot w$ mang giá trị từ $-\infty$ đến $+\infty$.
- Để đưa về một xác suất $[0, 1]$, ta dùng hàm **sigmoid** $\sigma(x)$:

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)} \quad (5.15)$$

- Vì tổng xác suất của hai biến cố (c là context word, và c không là context word) phải bằng 1. Nên ta ước lượng xác suất c không phải là real context word của w như sau:

$$P(-|w, c) = 1 - P(+|w, c) = \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)} \quad (5.16)$$

Mathematics of the Classifier: Multiple Context Words

- Phương trình 5.15 chỉ tính xác suất cho một từ, nhưng trong context window có rất nhiều context words.
- Skip-gram đưa ra giả định: tất cả các context words là độc lập với nhau. Do đó, ta tính xác suất đồng thời bằng cách nhân các xác suất lại với nhau:

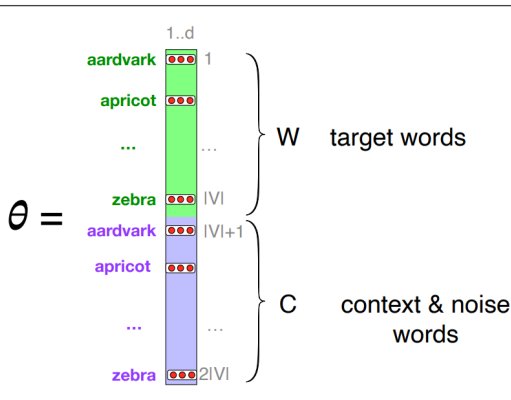
$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w) \quad (5.17)$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w) \quad (5.18)$$

- Để tính toán xác suất, ta cần các vector embeddings cho mọi từ trong vocabulary. Skip-gram lưu trữ hai embeddings cho mỗi từ: một vector khi nó đóng vai trò là target word và một vector khi nó là context word.

Parameters of Skip-gram

Skip-gram lưu trữ hai embeddings cho mỗi từ: một khi từ đó là target word và một khi nó là context word. Mỗi từ được biểu diễn bằng một vector hàng kích thước d . Tổng tham số θ là ma trận $2|V| \times d$ nối từ hai ma trận $\mathbf{W}$ và $\mathbf{C}$.



Learning Skip-gram: Initialization & Positive Examples

Khởi tạo thuật toán:

- **Input:** Một tập văn bản và kích thước từ vựng N .
- Thuật toán gán ngẫu nhiên một embedding vector cho mỗi từ.

Trích xuất các Positive Examples:

- Xét câu ví dụ với từ đích $w = \text{apricot}$ và window $L = \pm 2$:
*... lemon, a [tablespoon_{c1} of_{c2} **apricot**_w jam_{c3}, a_{c4}] pinch ...*
- Kết quả ta thu được 4 positive training instances (w, c_{pos}):
(apricot, tablespoon), (apricot, of), (apricot, jam), (apricot, a)

Learning Skip-gram: Negative Sampling (SGNS)

Các nguyên tắc tạo Negative Examples:

- Để huấn luyện binary classifier, ta cần các ví dụ âm. SGNS sử dụng nhiều negative examples hơn positive examples, tỷ lệ này được quyết định bởi tham số k .
- Với mỗi cặp (w, c_{pos}) , ta tạo ra k mẫu âm (w, c_{neg}) .
- Noise word (c_{neg}): Một từ bốc ngẫu nhiên từ từ điển, với điều kiện không được trùng với target word w .

Ví dụ minh họa với tham số $k = 2$:

Positive Examples +		Negative Examples -			
w	c_{pos}	w	c_{neg}	w	c_{neg}
apricot	tablespoon	apricot	aardvark	apricot	seven
apricot	of	apricot	my	apricot	forever
apricot	jam	apricot	where	apricot	dear
apricot	a	apricot	coaxial	apricot	if

Learning Skip-gram: Noise Word Distribution

- Nếu lấy mẫu theo xác suất không trọng số $P(w)$ (unweighted probability), các từ phổ biến như "*the*" sẽ được chọn liên tục, trong khi từ hiếm như "*aardvark*" hiếm khi được chọn.

Learning Skip-gram: Noise Word Distribution

- Nếu lấy mẫu theo xác suất không trọng số $P(w)$ (unweighted probability), các từ phổ biến như "*the*" sẽ được chọn liên tục, trong khi từ hiếm như "*aardvark*" hiếm khi được chọn.

Giải pháp khắc phục

Chọn noise words dựa trên **weighted unigram probability** $P_\alpha(w)$:

$$P_\alpha(w) = \frac{\text{count}(w)^\alpha}{\sum_{w'} \text{count}(w')^\alpha} \quad (5.19)$$

Learning Skip-gram: Noise Word Distribution

- Trong thực tế, người ta thường thiết lập $\alpha = 0.75$, sử dụng $P_{\frac{3}{4}}(w)$. Sử dụng $\alpha = 0.75$ giúp tăng nhẹ xác suất cho các từ hiếm: $P_{\alpha}(w) > P(w)$.
- Ví dụ: Giả sử từ phổ biến có $P(a) = 0.99$ và từ hiếm có $P(b) = 0.01$:

$$P_{0.75}(a) = \frac{0.99^{0.75}}{0.99^{0.75} + 0.01^{0.75}} = 0.97$$

$$P_{0.75}(b) = \frac{0.01^{0.75}}{0.99^{0.75} + 0.01^{0.75}} = 0.03$$

→ Xác suất bị bốc trúng của từ hiếm b tăng từ 0.01 lên 0.03.

Learning Skip-gram: Training Goals

Với các ví dụ positive/negative và các embeddings khởi tạo, ta cần điều chỉnh các vector để:

- Tối đa hóa độ tương đồng (maximize similarity) của các cặp (w, c_{pos}) thuộc positive examples.
- Tối thiểu hóa độ tương đồng (minimize similarity) của các cặp (w, c_{neg}) thuộc negative examples.

Xét một cặp (w, c_{pos}) cùng với k noise words $c_{neg_1} \dots c_{neg_k}$, ta có hàm mất mát:

$$L(w, c_{pos}, c_{neg*}) = -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right]$$

Learning Skip-gram: Training Goals

Sử dụng tính chất của logarit, ta khai triển công thức:

$$\begin{aligned} L &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log(1 - P(+|w, c_{neg_i})) \right] \\ &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned} \quad (5.21)$$

Learning Skip-gram: Optimization

- Ta sử dụng Stochastic Gradient Descent để tối thiểu hóa hàm loss. Để có được gradient, ta tính đạo hàm riêng của phương trình 5.21 theo các vector embeddings khác nhau:

$$\frac{\partial L}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w \quad (5.22)$$

$$\frac{\partial L}{\partial c_{neg_i}} = [\sigma(c_{neg_i} \cdot w)]w \quad (5.23)$$

$$\frac{\partial L}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w)]c_{neg_i} \quad (5.24)$$

Learning Skip-gram: Optimization

Từ bước t sang $t + 1$ trong SGD, với learning rate η , ta cập nhật các vector embeddings:

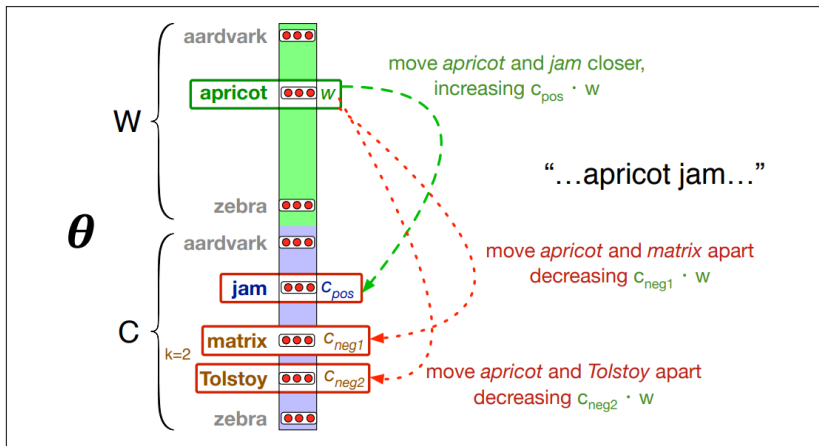
$$c_{pos}^{t+1} = c_{pos}^t - \eta[\sigma(c_{pos}^t \cdot w^t) - 1]w^t \quad (5.25)$$

$$c_{neg_i}^{t+1} = c_{neg_i}^t - \eta[\sigma(c_{neg_i}^t \cdot w^t)]w^t \quad (5.26)$$

$$w^{t+1} = w^t - \eta \left[[\sigma(c_{pos}^t \cdot w^t) - 1]c_{pos}^t + \sum_{i=1}^k [\sigma(c_{neg_i}^t \cdot w^t)]c_{neg_i}^t \right] \quad (5.27)$$

Learning Skip-gram: Optimization

Minh họa trực quan của một bước cập nhật các vector embeddings:



Learning Skip-gram: Final Embeddings & Window Size

Trích xuất word embeddings sau khi training xong:

- Để biểu diễn từ i sau khi huấn luyện, ta có 2 cách phổ biến:
 - 1 Biểu diễn từ i bằng vector tổng $w_i + c_i$.
 - 2 Bỏ đi ma trận C và chỉ sử dụng vector đích w_i làm embedding duy nhất.

Sự ảnh hưởng của context window size:

- Giống như các phương pháp đếm (như tf-idf), context window size ảnh hưởng trực tiếp đến hiệu suất của skip-gram embeddings.
- Kích thước này thường được xem là một hyperparameter và được tinh chỉnh thông qua các thử nghiệm trên tập devset.

Other Kinds of Static Embeddings: FastText & GloVe

1. FastText:

- Vấn đề của Word2vec: Không xử lý được các từ chưa biết và sự thưa thớt từ vựng (*word sparsity*) ở các ngôn ngữ phong phú hình thái.
- **Giải pháp:** Sử dụng mô hình **subword** dựa trên n-grams ký tự.
- Ví dụ: Với $n = 3$, từ *where* được biểu diễn thành $\langle where \rangle$ cộng với các n-grams: $\langle wh, whe, her, ere, re \rangle$.
- Embedding của một từ là tổng các embeddings của các n-grams cấu thành nên nó.

2. GloVe - Global Vectors

- Dựa trên việc nắm bắt thống kê toàn cục của tập dữ liệu (global corpus statistics).
- Sử dụng tỷ lệ xác suất từ ma trận đồng xuất hiện (word-word co-occurrence matrix).

- 1 Word2vec
- 2 Visualizing Embeddings**
- 3 Semantic properties of embeddings
- 4 Bias and Embeddings
- 5 Evaluating Vector Models

Vấn đề

Làm thế nào chúng ta có thể trực quan hóa một vector có 100 chiều?

- **Liệt kê từ lân cận:**

- Tìm và sắp xếp các từ trong từ điển theo độ tương đồng với từ đích w .
- Ví dụ: 7 từ gần nhất với *frog* khi sử dụng một tập hợp các embeddings được tính bằng thuật toán GloVe là: *frogs*, *toad*, *litoria*, *leptodactylidae*, *rana*, *lizard*, *eleutherodactylus*.

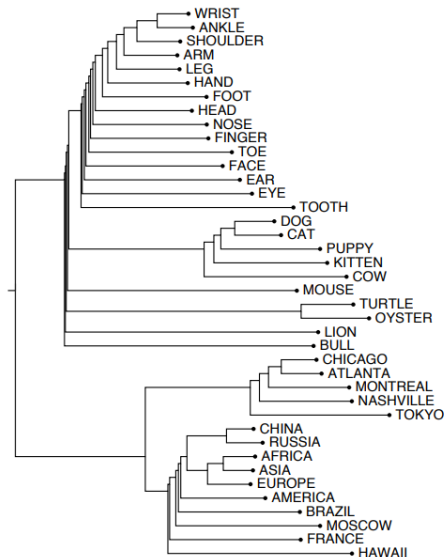
- **Hierarchical Clustering:**

- Sử dụng thuật toán phân cụm để nhóm các từ có vector tương đồng lại với nhau, tạo thành một cấu trúc cây.

- **Dimensionality Reduction:**

- Chuyển các vector 100 chiều xuống không gian thấp chiều hơn như **2D** hoặc **3D** để con người có thể quan sát.
- Đây là phương pháp phổ biến nhất hiện nay, thường sử dụng thuật toán **t-SNE**.

Visualizing Embeddings



Mục lục

- 1 Word2vec
- 2 Visualizing Embeddings
- 3 Semantic properties of embeddings**
- 4 Bias and Embeddings
- 5 Evaluating Vector Models

Giới thiệu về Semantic Properties

Các **thuộc tính ngữ nghĩa của embeddings** mô tả cách mà các vector này thể hiện:

- Sự giống nhau giữa các từ
- Quan hệ ngữ nghĩa và cú pháp
- Khả năng suy luận phép tương tự (analogy)
- Sự thay đổi nghĩa của từ theo thời gian

Những đặc tính này làm cho embeddings trở thành công cụ quan trọng trong các bài toán xử lý ngôn ngữ tự nhiên (NLP).

Context window xác định số lượng từ xung quanh một từ mục tiêu được sử dụng để xây dựng vector biểu diễn của từ đó.

Thông thường, cửa sổ ngữ cảnh có kích thước từ:

1 – 10 từ mỗi phía của từ mục tiêu (2 – 20 từ trong ngữ cảnh)

Ảnh hưởng của Context Window

Kích thước của context window ảnh hưởng trực tiếp đến loại quan hệ mà mô hình học được.

Context window nhỏ

- Thường học được các quan hệ **cú pháp**.
- Các từ xuất hiện trong ngữ cảnh giống nhau thường thuộc cùng loại từ.

Ví dụ: *run, walk, jump*

Context window lớn

- Học được các quan hệ **ngữ nghĩa**.
- Các từ liên quan về chủ đề hoặc ý nghĩa.

Ví dụ: *doctor, hospital, patient*

Ví dụ từ nghiên cứu

Levy và Goldberg (2014) đã chỉ ra rằng kích thước context window ảnh hưởng đến kết quả tìm từ tương tự.

Xét từ **Hogwarts** trong loạt truyện Harry Potter.

Với window nhỏ (± 2)

- Các từ giống nhất là tên của những trường học giả tưởng khác
- Ví dụ: Sunnydale, Evernight

Với window lớn (± 5)

- Các từ giống nhất liên quan trực tiếp đến thể giới Harry Potter
- Ví dụ: Dumbledore, Malfoy, half-blood

Context window quyết định loại quan hệ được học trong embeddings.

Types of Word Association

Embeddings có thể phân biệt hai loại liên hệ giữa các từ (Schütze & Pedersen, 1993):

First-order Co-occurrence

Hai từ **xuất hiện trực tiếp cùng nhau** trong văn bản.

Ví dụ:

U.S. + aircraft

Second-order Co-occurrence

Hai từ **không cần xuất hiện cạnh nhau** nhưng thường xuất hiện trong **các ngữ cảnh giống nhau**.

Ví dụ:

doctor $\approx$ physician

Analogy và Relational Similarity

Embeddings có khả năng biểu diễn các **quan hệ tương tự** giữa các từ.
Dạng bài toán thường gặp là:

$$a : b :: a' : b'$$

Ví dụ:

$$apple : tree :: grape : vine$$

Các mô hình embeddings có thể giải quyết bài toán này bằng cách thực hiện các phép toán vector trong không gian embedding.

Mô Hình Hình Bình Hành

Rumelhart và Abrahamson (1973) đề xuất **mô hình hình bình hành** để giải thích cách embeddings xử lý các bài toán analogy.

Ví dụ:

$$tree - apple + grape \approx vine$$

Quy trình:

- Tính vector quan hệ: $tree - apple$
- Áp dụng vector quan hệ đó cho từ mới: $grape$
- Tìm vector gần nhất trong không gian embeddings

Kết quả là từ *vine*, tạo thành cấu trúc hình bình hành.

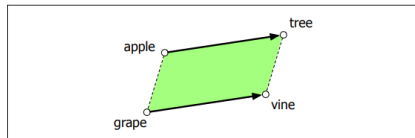


Figure 5.8 The parallelogram model for analogy problems (Rumelhart and Abrahamson, 1973): the location of *vine* can be found by subtracting *apple* from *tree* and adding *grape*.

Ví dụ Analogy Nổi Tiếng

Các mô hình embeddings hiện đại như Word2Vec và GloVe thể hiện khả năng giải analogy rất tốt.

Ví dụ nổi tiếng:

$$\textit{king} - \textit{man} + \textit{woman} \approx \textit{queen}$$

Ví dụ khác:

$$\textit{Paris} - \textit{France} + \textit{Italy} \approx \textit{Rome}$$

Các kết quả này cho thấy embeddings có thể học được nhiều loại quan hệ như:

- Quan hệ giới tính
- Quan hệ thủ đô - quốc gia
- Biến đổi hình thái của từ

Đối với bài toán analogy $a : b :: a' : b'$, mục tiêu là tìm từ b' sao cho vector của nó gần nhất với biểu thức:

$$b - a + a'$$

Công thức tổng quát:

$$b' = \arg \min_x distance(x, b - a + a')$$

Trong đó hàm distance có thể là:

- Euclidean distance
- Dot product
- Cosine similarity

Hạn Chế của Phương Pháp

Mặc dù phương pháp hình bình hành hoạt động tốt trong nhiều trường hợp, nó vẫn có một số hạn chế.

Ví dụ:

- potato : potatoes :: brown : ?

Mô hình có thể trả về:

- potato hoặc potatoes

thay vì kết quả mong đợi: browner

Nguyên nhân:

- Một số quan hệ ngôn ngữ không tuyến tính trong không gian vector
- Các từ có hình thái tương tự hoặc các từ phổ biến có thể ảnh hưởng mạnh đến phép đo khoảng cách.

Embeddings và Ngữ nghĩa Lịch sử

Embeddings có thể được dùng để nghiên cứu sự thay đổi nghĩa của từ theo thời gian (*semantic shift*).

- Huấn luyện các không gian embedding riêng cho từng giai đoạn thời gian
- So sánh vị trí vector của cùng một từ qua các thời kỳ
- Sự dịch chuyển của vector cho thấy sự thay đổi ngữ nghĩa

Ví dụ semantic shift:

- **gay**: vui vẻ, tươi sáng → đồng tính
- **broadcast**: gieo rải hạt → truyền phát (radio, TV)
- **awful**: đầy sự kính sợ → khủng khiếp

Mục lục

- 1 Word2vec
- 2 Visualizing Embeddings
- 3 Semantic properties of embeddings
- 4 Bias and Embeddings**
- 5 Evaluating Vector Models

Word Embeddings and Bias

1. Sự hấp thụ dữ liệu

- Word embeddings học ý nghĩa và quan hệ giữa các từ từ kho dữ liệu văn bản khổng lồ.
- **Mặt trái:** Chúng cũng tự động học luôn các định kiến xã hội (bias) tiềm ẩn trong dữ liệu đó.
- Các bias này có thể phản ánh những thành kiến về giới tính, chủng tộc, nghề nghiệp, v.v.

2. Ví dụ về analogy trong embeddings

Sự tương đồng hợp lý

king – man + woman $\approx$ queen

Phản ánh định kiến (bias)

computer programmer – man + woman $\approx$ homemaker

The two main types of harm caused by bias.

1. Allocational Harm (Tác hại phân bổ)

- Hệ thống phân bổ tài nguyên hoặc cơ hội không công bằng giữa các nhóm xã hội. Embeddings bias có thể khiến hệ thống tước đi cơ hội của một nhóm.
- **Ví dụ:** Hệ thống lọc CV tự động (AI tuyển dụng) đánh giá thấp hồ sơ của ứng viên nữ cho vị trí Kỹ sư vì mô hình liên kết từ khóa "engineer" hoặc "programmer" mạnh mẽ hơn với nam giới.

2. Representational Harm (Tác hại đại diện)

- Hệ thống thể hiện một nhóm xã hội theo cách tiêu cực, dập khuôn hoặc hạ thấp.
- **Ví dụ 1:** Hệ thống dịch máy tự động dịch "the doctor" thành "ông bác sĩ" và "the nurse" thành "cô y tá".
- **Ví dụ 2:** Mô hình phân tích cảm xúc (Sentiment Analysis) tự động đánh giá các câu văn có chứa tên của một nhóm dân tộc có sắc thái tiêu cực hơn, mặc dù ngữ cảnh hoàn toàn bình thường.

Minimize Bias in Word Embeddings

1. Mục tiêu cốt lõi

- Giảm thiểu hoặc loại bỏ các định kiến tồn tại trong embedding space.
- **Thách thức:** Phải thực hiện điều này nhưng vẫn giữ nguyên được ý nghĩa ngữ nghĩa và tính hữu dụng cốt lõi của từ.

2. Kết quả kỳ vọng

Loại bỏ định kiến nghề nghiệp

doctor – man + woman $\neq$ nurse

(Mô hình không tự động gán nghề y tá cho nữ giới, hướng đến kết quả trung lập hơn)

Minimize Bias in Word Embeddings

Các hướng tiếp cận chính

• **Biến đổi embedding space (Post-processing):**

- Áp dụng các phép biến đổi toán học lên các vector đã được huấn luyện.
- Mục đích là để triệt tiêu các thành phần đại diện cho định kiến (ví dụ: định kiến nam nữ).

• **Thay đổi quy trình huấn luyện (Training procedure):**

- Can thiệp trực tiếp vào dữ liệu đầu vào (ví dụ: tinh chỉnh, làm sạch hoặc cân bằng lại dữ liệu).
- Điều chỉnh thuật toán và hàm mục tiêu để ngăn mô hình học các định kiến ngay từ đầu.

Lưu ý

- **Debiasing chỉ giảm bias**
- **Không thể loại bỏ hoàn toàn**

Details of methods to reduce bias

1. Post-processing

- **Xác định hướng:** Tìm *gender direction* (hướng đại diện cho giới tính) thông qua các vector của các cặp từ chuẩn (ví dụ: he - she, man - woman).
- **Trung lập hóa (Neutralize):** Loại bỏ thành phần giới tính khỏi vector của các từ vốn mang ý nghĩa trung lập (như doctor, engineer).
- **Ví dụ toán học:** Vector mới của từ "doctor" được tính bằng cách trừ đi hình chiếu của nó trên hướng giới tính.

$$\text{doctor}' = \text{doctor} - \text{projection}_{\text{gender}}(\text{doctor})$$

2. Training procedure

- **Điều chỉnh hàm mục tiêu:** Thêm thành phần *regularization* (điều chuẩn) vào hàm mất mát.
- **Cơ chế phạt (Penalty):** Mô hình sẽ bị "phạt" nếu trong quá trình học, nó đặt các từ trung lập quá gần với *gender direction*.

- 1 Word2vec
- 2 Visualizing Embeddings
- 3 Semantic properties of embeddings
- 4 Bias and Embeddings
- 5 Evaluating Vector Models**

Evaluating Vector Models

Có hai phương pháp chính:

- **1. Extrinsic evaluation** (Đánh giá ngoại tại)
- **2. Intrinsic evaluation** (Đánh giá nội tại)

Extrinsic evaluation

- **Khái niệm:** Embeddings được tích hợp và sử dụng trong một tác vụ NLP thực tế.
- **Ví dụ các tác vụ:**
 - Sentiment analysis (Phân tích cảm xúc).
 - Machine translation (Dịch máy).
 - Named entity recognition (Nhận dạng thực thể có tên).
- **Tiêu chí:** Nếu việc áp dụng embeddings giúp tăng hiệu suất của hệ thống trong các tác vụ này, thì chúng được coi là những embeddings tốt.

Intrinsic Evaluation: Word Similarity

1. Intrinsic evaluation

- Đánh giá **trực tiếp** chất lượng của các embeddings mà không cần thông qua một tác vụ NLP nào khác.
- Phương pháp phổ biến nhất là dựa trên **word similarity** (độ tương đồng từ vựng).

2. Quy trình đánh giá

- 1 Con người tiến hành cho điểm mức độ tương đồng (similarity) giữa các cặp từ.
- 2 Hệ thống tính toán *cosine similarity* cho các cặp từ đó bằng các vector embeddings.
- 3 Đo lường hệ số tương quan (correlation) giữa kết quả đánh giá của con người và kết quả tính toán của mô hình.

3. Các bộ dữ liệu (Datasets) phổ biến

- WordSim-353
- SimLex-999
- TOEFL synonym test

The Randomness

1. Nguồn gốc của sự ngẫu nhiên

- Quá trình **random initialization** (khởi tạo trọng số ngẫu nhiên ban đầu).
- Kỹ thuật **negative sampling** (lấy mẫu âm ngẫu nhiên trong quá trình huấn luyện).

2. Hệ quả

- Việc huấn luyện mô hình trên **cùng một tập dữ liệu** nhiều lần vẫn có thể cho ra các bộ embeddings và kết quả đánh giá hoàn toàn khác nhau.

3. Giải pháp

Giải pháp ổn định đánh giá

- Huấn luyện nhiều bộ embeddings độc lập.
- Sử dụng phương pháp **bootstrap sampling** (lấy mẫu có hoàn lại) trên các tập dữ liệu kiểm tra.
- Lấy trung bình kết quả để có được đánh giá cuối cùng chính xác và đáng tin cậy hơn.